计算机组成与结构
—计算机中的数据表示3

李瑞　　教授
蒋志平　副教授
计算机科学与技术学院

温故 —关于数制表示

- 8位原码的表达范围？ -127， 0， 127的8位原码是?
- 8位反码的表达范围？ -127， 0， 127的8位反码是?
- 8位补码的表达范围？ -127， 0， 127的8位补码是?
- 8位移码的表达范围？ -127， 0， 127的8位移码是?
- 计算机中，如何表示实数?
- 浮点数的规格化是干嘛的？有嘛用?
- 0.001[补]如何规格化?
- IEEE 754格式？相比一般的规格化浮点，有什么特殊点?

本章学习要点

- 数字的表示
  - 进制编码基础
  - 整数及小数进制转换（给人看）
  - 整数及小数在计算机里的表示（给机器看）
  - 有符号号的原、反、补、移码
  - 浮点表示
    - IEEE754
  - BCD
- 非数字的表示
  - 字符表示
    - ASCII
    - 汉字
    - Unicode
  - 校验码
    - 奇偶校验
    - CRC

BCD编码

BCD编码

- BCD——Binary Coded Decimal，“用二进制编码的十进制”
- 它有两大类变形（Variant）：有权(Weighted)、无权(Un-weighted)

十进制数	8421 码	5421 码	2421 码	5211 码	84-2-1 码	4311 码
0	0000	0000	0000	0000	0000	0000
1	0001	0001	0001	0001	0111	0001
2	0010	0010	0010	0011	0110	0011
3	0011	0011	0011	0101	0101	0100
4	0100	0100	0100	0111	0100	1000
5	0101	1000	1011	1000	1011	0111
6	0110	1001	1100	1010	1010	1011
7	0111	1010	1101	1100	1001	1100
8	1000	1011	1110	1110	1000	1110
9	1001	1100	1111	1111	1111	1111

BCD编码

- BCD——Binary Coded Decimal，“用二进制编码的十进制”
- 它有两大类变形（Variant）：有权(Weighted)、无权(Un-weighted)

十进制数	余 3 码	格雷码（1）	格雷码（2）	格雷码（3）
0	0011	0000	0000	0000
1	0100	0001	0001	0100
2	0101	0011	0011	0110
3	0110	0010	0010	0010
4	0111	0110	0110	1010
5	1000	0111	1110	1011
6	1001	0101	1010	0011
7	1010	0100	1000	0001
8	1011	1100	1100	1001
9	1100	1000	0100	1000

BCD编码

- Why BCD？
  - 绝对想像不到的历史原因：
    - 最早的计算机，是十进制的！
      - ENIAC（1945）
      - IBM 1400系列（late-1950s）
      - IBM 1620系列（mid-1950s）
      - IBM 7070系列（early-1960s）
      - IBM System/360系列（1964）之后才转向全面二进制
- 后期的计算机出于兼容目的保留BCD代码



IBM 1401 Data Processing System



IBM 1602 Industrial Process Control System



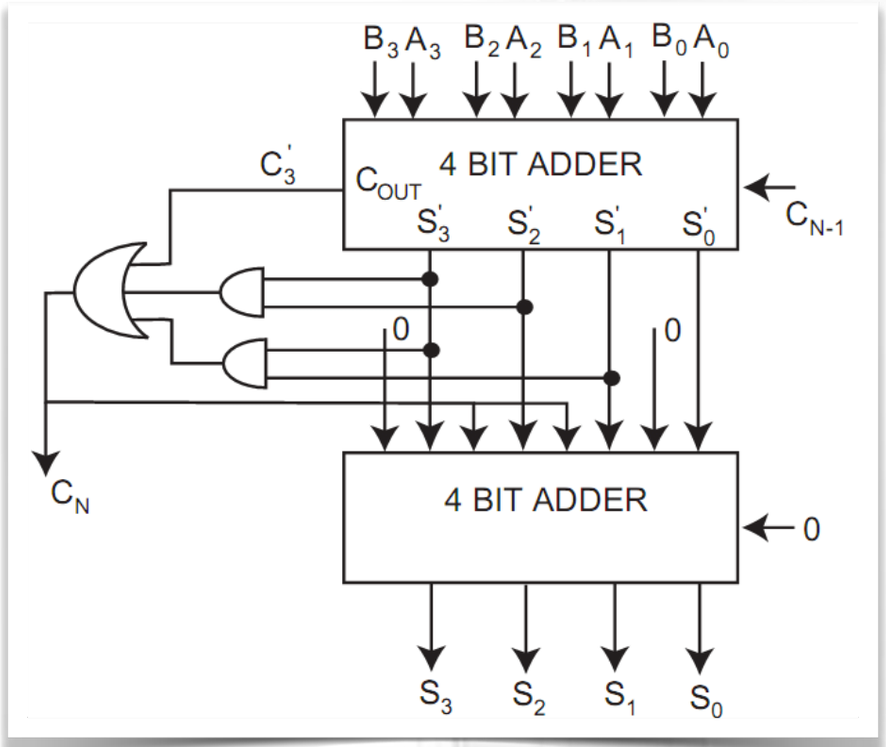
IBM 7070 Series

BCD编码

- then, why not BCD now?
  - 基于BCD的计算麻烦、慢
  - 算个加法，你就明白了

- 二进制加法，一步搞定

$$(9)_D \quad +(8)_D \quad = (17)_D$$
$$(1001)_B \quad +(1000)_B \quad = (10001)_B$$



BCD加法器= 二进制加法器 + 额外电路

- BCD版本加法，还要再+06H

$$(10001)_B \quad +(0110)_B \quad = (00010111)_B \Rightarrow \quad 0001 \quad 0111$$
$$(17)_D \quad +(6)_D \quad = (23)_D \Rightarrow \quad 1 \quad 7$$

非数值数据编码

非数值数据编码(Non-numeric data representation)

- 文字
- 声音
- 图像
- 视频

文字编码与表示

文本编码与表示包含三个方面：

- 输入编码
  - 拼音、五笔、区位
- 字符编码（内码）
  - ASCII？
  - GB2312? BIG5 ?GB10830?
  - Unicode?
    - UTF8/16/32?
- 显示
  - 点阵字体?
  - 矢量字体?
    - 宋体?
    - 黑体?

输入码 v.s 字符编码

		计	算	机	组	成	与	结	构
输入码	拼音*	ji	sua	ji	zuc	che	yu	jie	gou
	五笔	yfh	tha	sm	xeg	dn	gng	xfk	sqc
编码	ASCII	-	-	-	-	-	-	-	-
	GB2312	bcc6	cbe3	bbfa	d7e9	b3c9	d3eb	bde1	b9b9
	BIG5	-	bae2	c9f3	-	a6a8	c94f	-	ccdb
	GBK	bcc6	cbe3	bbfa	d7e9	b3c9	d3eb	bde1	b9b9
	Unicode	8ba1	7b97	673a	7ec4	6210	4e0e	7ed3	6784

点阵字体 v.s 矢量字体

	计	算	机	组	成	与	结	构
黑体像素	计	算	机	组	成	与	结	构
黑体	计	算	机	组	成	与	结	构
仿宋	计	算	机	组	成	与	结	构
微软雅黑	计	算	机	组	成	与	结	构

文字编码与表示—ASCII

ASCII（/ˈæskiː/, ASS-kee），American Standard Code for Information Interchange, 美国信息交换标准代码，1963年发布，对应ISO 646标准。

- 7bit（存入1Byte），编码128个字符：
  - 32个控制字符
  - 空格(Space)，删除(DEL)
  - 数字0～9，编码为30H加上数字值（0 for 0x30H, 9 for 0x39H）
  - 52个英文字母
    - 大写字母，编码为40H加上字母顺序
    - 小写字母，编码为60H加上字母顺序
  - 32个标点符号
  - 共94个可打印符号（0x21 - 0x7e)

	$b_6\,b_5\,b_4$ / $b_3\,b_2\,b_1\,b_0$	0	1	2	3	4	5	6	7
		000	001	010	011	100	101	110	111
0	0000	NUL	DLE	Sp	0	@	P	`	p
1	0001	SOH	DC1	!	1	A	Q	a	q
2	0010	STX	DC2	"	2	B	R	b	r
3	0011	ETX	DC3	#	3	C	S	c	s
4	0100	EOT	DC4	$	4	D	T	d	t
5	0101	ENQ	NAK	%	5	E	U	e	u
6	0110	ACK	SYN	&	6	F	V	f	v
7	0111	BEL	ETB	'	7	G	W	g	w
8	1000	BS	CAN	(	8	H	X	h	x
9	1001	HT	EM	)	9	I	Y	i	y
A	1010	LF	SUB	*	:	J	Z	j	z
B	1011	VT	ESC	+	;	K	[	k	{
C	1100	FF	FS	,	<	L	\	l	|
D	1101	CR	GS	-	=	M	]	m	}
E	1110	SO	RS	.	>	N	^	n	~
F	1111	SI	US	/	?	O	_	o	DEL

文字编码与表示—Extended ASCII

- ASCII编码，只覆盖了英语字符（or 英语国家）
- Extended ASCII, ISO 8895-*n*标准，覆盖了拉丁语系字符（or 欧洲、部分西亚国家）
- 将“高”128位用于编码，
  - 但只用0xa0 - 0xff，扩展了96个字符
- 为什么0x80-0x9f不用?
  - 继承ASCII 7-bit编码~
- ISO 8895-*n*标准更新过多轮，ISO 8895-1最具代表性，常称为ISO Latin-1（1987）
- 最新标准是ISO 8895-16，1998年，此后放弃更新，因为unicode来了

ISO/IEC 8859-1

	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9	_A	_B	_C	_D	_E	_F
0_0																
1_16																
2_32	SP 0020	! 0021	" 0022	# 0023	$ 0024	% 0025	& 0026	' 0027	(0028	) 0029	* 002A	+ 002B	, 002C	− 002D	. 002E	/ 002F
3_48	0 0030	1 0031	2 0032	3 0033	4 0034	5 0035	6 0036	7 0037	8 0038	9 0039	: 003A	; 003B	< 003C	= 003D	> 003E	? 003F
4_64	@ 0040	A 0041	B 0042	C 0043	D 0044	E 0045	F 0046	G 0047	H 0048	I 0049	J 004A	K 004B	L 004C	M 004D	N 004E	O 004F
5_80	P 0050	Q 0051	R 0052	S 0053	T 0054	U 0055	V 0056	W 0057	X 0058	Y 0059	Z 005A	[005B	\ 005C	] 005D	^ 005E	_ 005F
6_96	` 0060	a 0061	b 0062	c 0063	d 0064	e 0065	f 0066	g 0067	h 0068	i 0069	j 006A	k 006B	l 006C	m 006D	n 006E	o 006F
7_112	p 0070	q 0071	r 0072	s 0073	t 0074	u 0075	v 0076	w 0077	x 0078	y 0079	z 007A	{ 007B	| 007C	} 007D	~ 007E	
8_128																
9_144																
A_160	NBSP 00A0	¡ 00A1	¢ 00A2	£ 00A3	¤ 00A4	¥ 00A5	¦ 00A6	§ 00A7	¨ 00A8	© 00A9	ª 00AA	« 00AB	¬ 00AC	SHY 00AD	® 00AE	¯ 00AF
B_176	° 00B0	± 00B1	² 00B2	³ 00B3	´ 00B4	µ 00B5	¶ 00B6	· 00B7	¸ 00B8	¹ 00B9	º 00BA	» 00BB	¼ 00BC	½ 00BD	¾ 00BE	¿ 00BF
C_192	À 00C0	Á 00C1	Â 00C2	Ã 00C3	Ä 00C4	Å 00C5	Æ 00C6	Ç 00C7	È 00C8	É 00C9	Ê 00CA	Ë 00CB	Ì 00CC	Í 00CD	Î 00CE	Ï 00CF
D_208	Ð 00D0	Ñ 00D1	Ò 00D2	Ó 00D3	Ô 00D4	Õ 00D5	Ö 00D6	× 00D7	Ø 00D8	Ù 00D9	Ú 00DA	Û 00DB	Ü 00DC	Ý 00DD	Þ 00DE	ß 00DF
E_224	à 00E0	á 00E1	â 00E2	ã 00E3	ä 00E4	å 00E5	æ 00E6	ç 00E7	è 00E8	é 00E9	ê 00EA	ë 00EB	ì 00EC	í 00ED	î 00EE	ï 00EF
F_240	ð 00F0	ñ 00F1	ò 00F2	ó 00F3	ô 00F4	õ 00F5	ö 00F6	÷ 00F7	ø 00F8	ù 00F9	ú 00FA	û 00FB	ü 00FC	ý 00FD	þ 00FE	ÿ 00FF

文字编码与表示—Extended Unix Code (EUC)

- ASCII + Extended ASCII 只有190个可打印字符，太少！
- EUC是非常重要的进步：可变长，可使用最多4个字节给字符编码！
- 继承ASCII 7-bit编码，每个字节94个可编码空间：
  - 双字节=>$94^2 = 8836$，三字节=>94^3，四字节=>94^4
- 足够应付基本的CJK字符集了
  - 中国，GB2312-80，6783个汉字，编码格式为EUC-CN，1980年
    - GB10830，23940个汉字，修改的EUC-CN (126x191)
  - 日本，JIS X 0208，6879个字符（6355个汉字），编码格式为EUC-JP
  - 韩国，KS X 1001，8227个字符（4888个汉字），编码格式为EUC-KR
- 国际标准ISO 2022
- 不足的是，不同字符集的编码是重叠的...

文字编码与表示—大一统Unicode

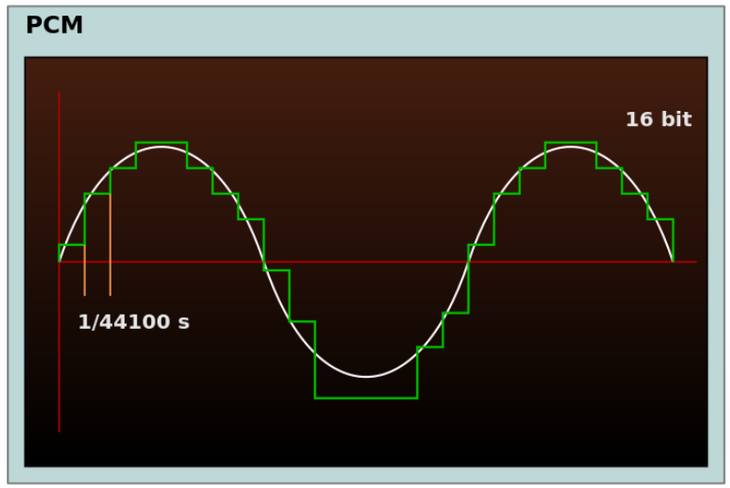
- Unicode，给每一个字符一个唯一编码，不论语言、平台、程序…
- 将字符唯一编号（Unicode Code Point）与传输用编码方式（UTF-8/16）区分
- 每个字符都有一个32位Code Point，对应一个最长4字节编码
- Unicode 13.0 包含143924个字符
- 对应国际标准ISO 10646
- 大统一
  - 主流平台、主流编程语言的核心编码
    - Windows / Linux / macOS
    - Java / .Net / Python 3.x / C++
  - 99% Web内容

Character	Code Point	UTF-16	UTF-8
a	U+0061	0061	61
ä	U+00E4	00E4	C3 A0
σ	U+03C3	03C3	CF 83
א	U+05D0	05D0	D7 90
٣	U+0663	0663	D9 A3
カ	U+30AB	30AB	E3 82 AB
退	U+9000	9000	E9 80 80
𠮷	U+21BC1	D846 DFC1	F0 A1 AF 81

声音的编码与表示

声音的编码包含两方面：

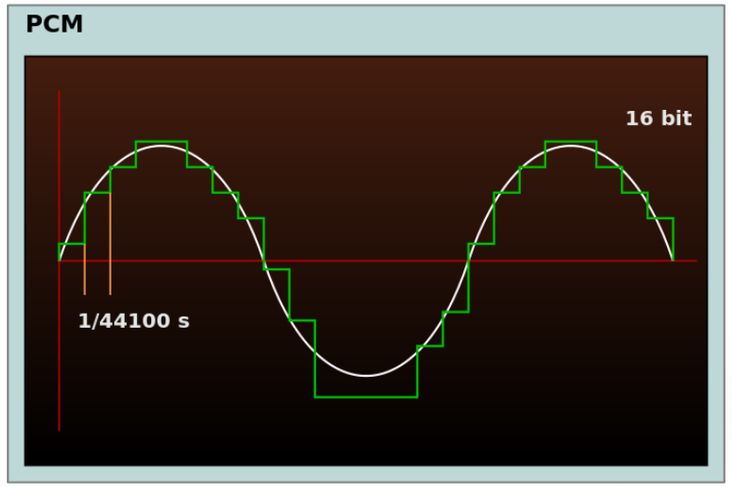
- 编码模式
  - PCM / DSD
- 存储/传输方式
  - 非压缩
    - .wav / .AIFF / raw PCM
  - 压缩
    - 无损
      - .flac / .ape / .wv
    - 有损
      - .mp3 / .ACC



声音的编码与表示

声音的编码包含两方面：

- 编码模式
  - PCM / DSD
- 存储/传输方式
  - 非压缩
    - .wav / .AIFF / raw PCM
  - 压缩
    - 无损
      - .flac / .ape / .wv
    - 有损
      - .mp3 / .ACC



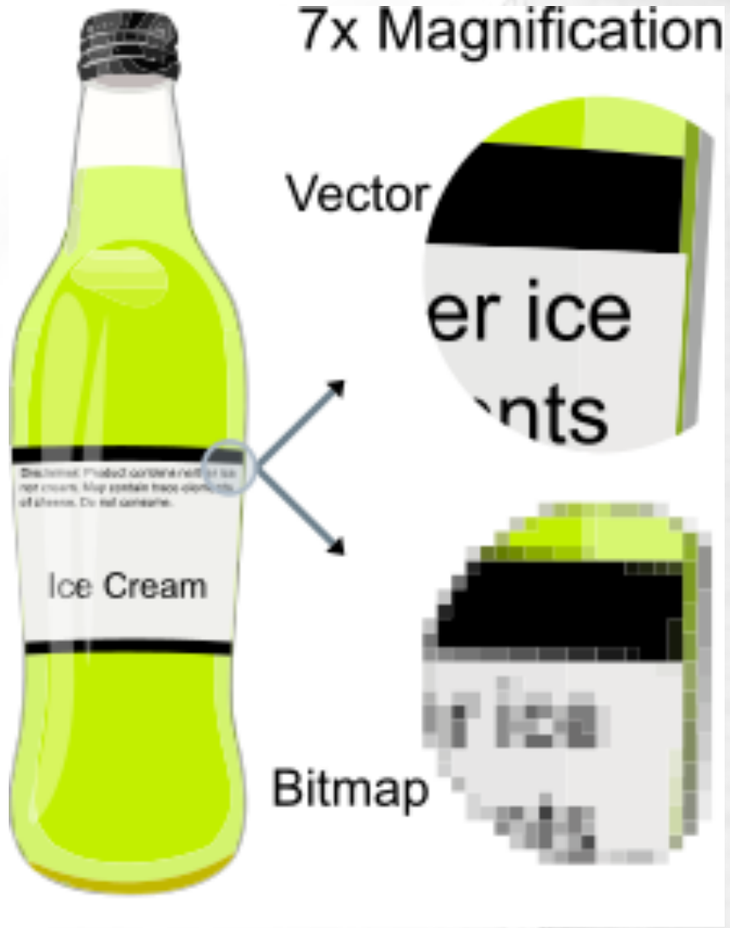
额外唠叨，劝青年莫交Hi-Fi智商税：

- “煲机”
- “推力”
- “声音冷/暖”
- “前端功放”
- “胆管”
- “黑胶唱片”
- “镀金线”
- “声场”
- “频响不能反应耳机的全部特性”

图像/视频的编码与表示

图像/视频编码包含三个方面：

- 数据类型
  - bitmap / vector
- 数据模式
  - Mono / RGB / RGB-D
- 存储格式
  - H.265 (HEVC / HEIF)
  - H.264
  - JPEG / Motion JPEG



位图 v.s 矢量

检错与纠错码

检错与纠错码

- 计算机内部是会出错的，网络也是会出错的！
  - 元器件故障；
  - 存储介质；
  - 通信过程中的噪声干扰...
- 如何避免错误：
  - 提高硬件或网络可靠性；
  - 采取数据检错和校正措施
    - 使用校验位，发现错误，甚至改错

检错与纠错码

其实是通信原理中，非常核心的一部分内容，也是非常庞大的专门学科: Coding Theory …

简单介绍以下三种：



- 检错码(Error Detection)：
  - 奇偶校验（Parity Check）
- 纠错码(Error Correction)：
  - 海（汉）明码（Hamming Code）
  - 循环冗余校验（CRC）

奇偶校验(Parity Check)

- 最简单的错误检测编码(Error Detection Coding)
  - 只知道错了，不知道哪一位错了
- 两种具体方法：奇校验和偶校验

- 编码方法（数据发送端）：
  - 生成校验位：
    - 偶校验：若包含奇数个1则置1，否则0
    - 奇校验：若包含偶数个1则置1，否则0
  - 将校验位插入数据

- 检测方法（数据接收端）：
  - 偶校验：若数据（含校验位）包含偶数个1，则正确；
  - 奇校验：若数据（含校验位）包含奇数个1，则正确；
  - 将校验位去除

7位数据	1的个数	8位数据（含校验位）	
		偶校验	奇检验
0000000	0	00000000	10000000
1010001	3	11010001	01010001
1101001	4	01101001	11101001
1111111	7	11111111	01111111

奇/偶检验编码

偶校验	结果	奇检验	结果
00000000	OK	10001000	Error
11010001	OK	01010001	OK
01100001	Error	11101001	OK
11110111	Error	01111011	Error

奇/偶检验错误检测

小练习

- 用奇/偶校验编码原码2024?
- 用奇偶校验验证(01010101)b

奇偶校验

- 缺点：
  - 只能检测奇数个错误
  - 不能纠错
- 优点：
  - 开销小
  - 常用于校验1字节长的数据：通常1字节长的数据编码发生错误时，1位出错的概率较大，两位以上同时出错的概率非常小。
- 常用于
  - 存储器读写校验
  - 按字节传输过程中的数据校验

海明码(Hamming Coding)

- 可以发现一定数量的错误的位置，改错。
- 由Richard Hamming于1950年提出。
- 主要用于存储器数据的校验与纠正
- 核心基于奇偶校验
- 校正能力与额外加入的奇偶校验位的数目线性相关
- 适用于最有可能发生随机错误的系统。
  - 每一位的出错概率相同；
  - 每一位与其它位是否出错没有任何关联。

海明码(Hamming Coding)

海明码编码过程：

1. 确定校验位数
2. 确定校验位、数据位位置
3. 确定每个校验位覆盖的bit位
4. 计算每个检验值

如何确定校验位和数据位位置?

- 校验位放在$2^n, n \in \mathcal{N}$的位置
- 数据位依次顺延排列
  - 如下图所示

Bit position	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Encoded data bits	p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8	d9	d10	d11	p16	d12	d13	d14	d15

海明码(Hamming Coding)

海明码编码过程：

1. 确定校验位数
2. 确定校验位、数据位位置
3. 确定每个校验位覆盖的bit位
4. 计算每个检验值

每个校验位覆盖哪些bit位?

- p1覆盖位置编码为xxxx1的
- p2覆盖位置编码为xxx1x的
- p3覆盖位置编码为xx1xx的
- p4覆盖位置编码为x1xxx的
- p5覆盖位置编码为1xxxx的
- ...

如下图中红色覆盖的位置

Bit position		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	
Encoded data bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8	d9	d10	d11	p16	d12	d13	d14	d15	
Parity bit coverage	p1	✗		✗		✗		✗		✗		✗		✗		✗		✗		✗		
	p2		✗	✗			✗	✗			✗	✗			✗	✗			✗	✗		
	p4				✗	✗	✗	✗					✗	✗	✗	✗					✗	
	p8								✗	✗	✗	✗	✗	✗	✗	✗						
	p16																✗	✗	✗	✗	✗	

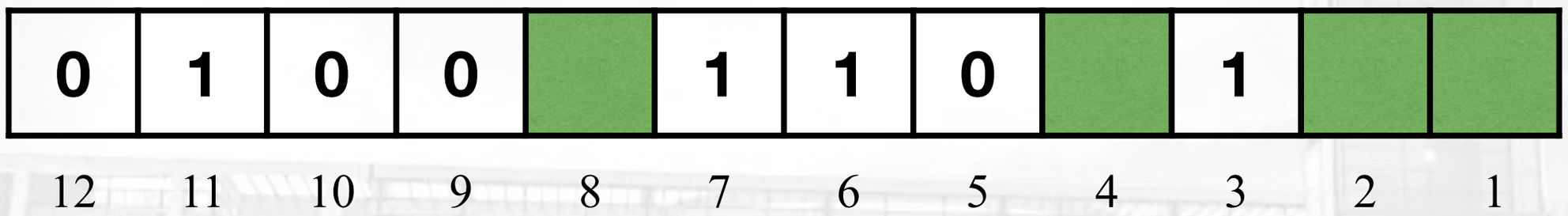
...

海明码(Hamming Coding)

【例 】利用上述构造海明码的步骤，采用偶校验，对8位ASCII字符“M”进行编码（最高位为0）。人为地引入一个1位错误，说明如何找出这个错误。

【解】字符“M”的ASCII编码为01001101。

1、确定所需校验位的数目



海明码(Hamming Coding)

【例 】利用上述构造海明码的步骤，采用偶校验，对8位ASCII字符“M”进行编码（最高位为0）。人为地引入一个1位错误，说明如何找出这个错误。

【解】字符“M”的ASCII编码为01001101。

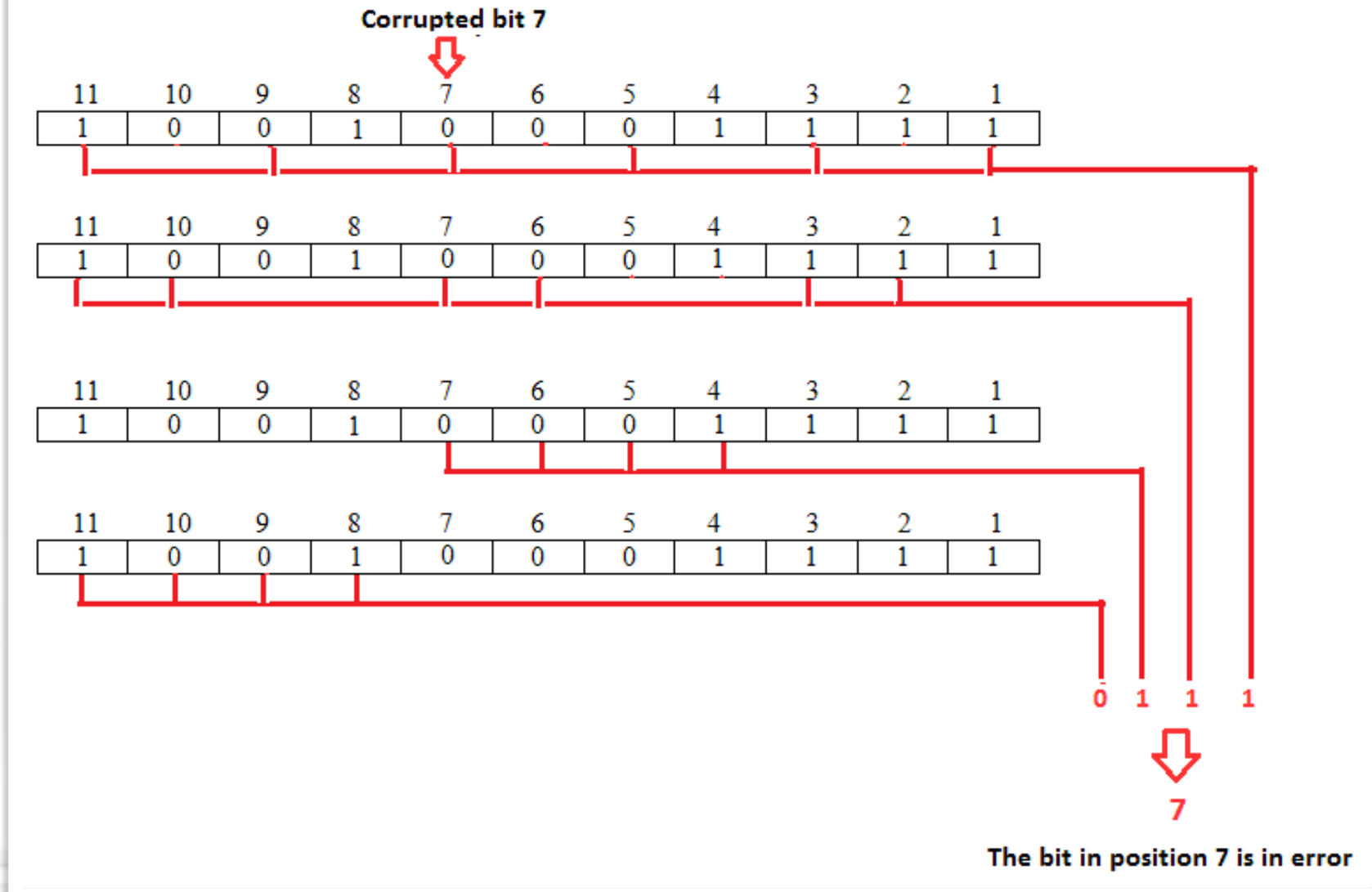
2、依次给出校验值，得到最终编码010011100101

0	1	0	0		1	1	0		1		1
0	1	0	0		1	1	0		1	0	1
0	1	0	0		1	1	0	0	1		1
0	1	0	0	1	1	1	0		1		1
12	11	10	9	8	7	6	5	4	3	2	1

海明码(Hamming Coding)

海明码接收检测过程：

1. 使用与发送端相同的海明码参数
2. 计算每个校验位p0, p1, p2, p3…
3. …p4p3p2p1p0指示了错误位！



Hamming Code

【例 】利用上述构造海明码的步骤，采用偶校验，对8位ASCII字符“M”进行编码（最高位为0）。人为地引入一个1位错误，说明如何找出这个错误。

【解】

字符“M”的ASCII编码为01001101。 假定H_0～H_3均为0，计算P_i：

$P_0 = D_6 \oplus D_4 \oplus D_3 \oplus D_1 \oplus D_0 \oplus H_0 = 1 \oplus 0 \oplus 1 \oplus 0 \oplus 1 \oplus 0 = 1 = H_0$

$P_1 = D_6 \oplus D_5 \oplus D_3 \oplus D_2 \oplus D_0 \oplus H_1 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 1 \oplus 0 = 0 = H_1$

$P_2 = D_7 \oplus D_3 \oplus D_2 \oplus D_1 \oplus H_2 = 0 \oplus 1 \oplus 1 \oplus 0 \oplus 0 = 0 = H_2$

$P_3 = D_7 \oplus D_6 \oplus D_5 \oplus D_4 \oplus H_3 = 0 \oplus 1 \oplus 0 \oplus 0 \oplus 0 = 1 = H_3$

∴ ASCII字符“M”对应的海明码编码字为：

$C = D_7 D_6 D_5 D_4 H_3 D_3 D_2 D_1 H_2 D_0 H_1 H_0 = 010011100101$

Hamming Code

【例 】利用上述构造海明码的步骤，采用偶校验，对8位ASCII字符“M”进行编码（最高位为0）。人为地引入一个1位错误，说明如何找出这个错误。

【解】在接收端，根据接收到的编码字求得校验方程P_i（i＝0～4），校验方程的编码值P($P=P_4P_3P_2P_1P_0$)可作如下解释：

(1) 如果P各位全是0，则表示接收到的编码字中没有错误。

(2) 如果P中多位为1，则表示有一个数据位出错，且P的值即出错位在编码字中的位置。纠错时只需将出错位取反即可。

(3) 如果P中有且仅有一位为1，则表示某一校验位出错，不需要纠正。

Hamming Code

【例 】利用上述构造海明码的步骤，采用偶校验，对8位ASCII字符“M”进行编码（最高位为0）。人为地引入一个1位错误，说明如何找出这个错误。

【解】接收端：如果接收到的编码字没有错误，接收到的编码字C′＝C＝010011100101，则

$P_0=D_6\oplus D_4\oplus D_3\oplus D_1\oplus D_0\oplus H_0=1\oplus 0\oplus 1\oplus 0\oplus 1\oplus 1=0$

$P_1=D_6\oplus D_5\oplus D_3\oplus D_2\oplus D_0\oplus H_1=1\oplus 0\oplus 1\oplus 1\oplus 1\oplus 0=0$

$P_2=D_7\oplus D_3\oplus D_2\oplus D_1\oplus H_2=0\oplus 1\oplus 1\oplus 0\oplus 0=0$

$P_3=D_7\oplus D_6\oplus D_5\oplus D_4\oplus H_3=0\oplus 1\oplus 0\oplus 0\oplus 1=0$

$P=P_3P_2P_1P_0=0000$，没有错误。

Hamming Code

【例 】利用上述构造海明码的步骤，采用偶校验，对8位ASCII字符“M”进行编码（最高位为0）。人为地引入一个1位错误，说明如何找出这个错误。

【解】接收端：如果接收到的编码字的数据位D_5出错,接收到的编码字$C'=$

01101110101，则

$P_0=D_6\oplus D_4\oplus D_3\oplus D_1\oplus D_0\oplus H_0=1\oplus 0\oplus 1\oplus 0\oplus 1\oplus 1=0$

$P_1=D_6\oplus D_5\oplus D_3\oplus D_2\oplus D_0\oplus H_1=1\oplus 1\oplus 1\oplus 1\oplus 1\oplus 0=1$

$P_2=D_7\oplus D_3\oplus D_2\oplus D_1\oplus H_2=0\oplus 1\oplus 1\oplus 0\oplus 0=0$

$P_3=D_7\oplus D_6\oplus D_5\oplus D_4\oplus H_3=0\oplus 1\oplus 1\oplus 0\oplus 1=1$

$P=P_3P_2P_1P_0=1010$,

编码字的**第十位(数据位D_5)发生错误**，将该位取反即可纠错。

Hamming Code

【例 】利用上述构造海明码的步骤，采用偶校验，对8位ASCII字符“M”进行编码（最高位为0）。人为地引入一个1位错误，说明如何找出这个错误。

【解】接收端：如果接收编码字的校验位H_2出错，接收到的编码字$C'=010011101101$，则

$P_0=D_6\oplus D_4\oplus D_3\oplus D_1\oplus D_0\oplus H_0=1\oplus 0\oplus 1\oplus 0\oplus 1\oplus 1=0$

$P_1=D_6\oplus D_5\oplus D_3\oplus D_2\oplus D_0\oplus H_1=1\oplus 0\oplus 1\oplus 1\oplus 1\oplus 0=0$

$P_2=D_7\oplus D_3\oplus D_2\oplus D_1\oplus H_2=0\oplus 1\oplus 1\oplus 0\oplus 1=1$

$P_3=D_7\oplus D_6\oplus D_5\oplus D_4\oplus H_3=0\oplus 1\oplus 0\oplus 0\oplus 1=0$

$P=P_3P_2P_1P_0=0100$,

编码字的第四位(校验位H_2)发生错误，无需纠错。

小练习

假设你有一个 8 位的数据字节为 11001101。使用编码对其进行编码。

几个小练习

假设你有一个 8 位的数据字节为 11001101。使用 Hamming编码对其进行编码。

步骤：

1. 确定校验位的位置。

   在 12 位的 Hamming 编码中，位置 1、2、4、8 是校验位的位置，而位置 3、5、6、7、9、10、11、12 是数据位的位置。

2. 插入数据位。

   将数据位插入到对应的位置上，即 1、2、4、8 位保持不变，而第 3 位插入数据位 1，第 5 位插入数据位 1，第 6 位插入数据位 0，第 7 位插入数据位 0，第 9 位插入数据位 1，第 10 位插入数据位 1，第 11 位插入数据位 0，第 12 位插入数据位 1，得到 12 位的编码为 110101001101。

3. 计算校验位。

   对校验位进行计算，使其满足奇偶校验规则。计算方式为：

   - 校验位 1：对位置 3、5、7、9、11 进行奇偶校验，得到校验位 1 的值为 1。
   - 校验位 2：对位置 3、6、7、10、11 进行奇偶校验，得到校验位 2 的值为 0。
   - 校验位 4：对位置 5、6、7、12 进行奇偶校验，得到校验位 4 的值为 1。
   - 校验位 8：对位置 9、10、11、12 进行奇偶校验，得到校验位 8 的值为 1。

4. 替换校验位。

   将计算得到的校验位替换到对应的位置上，得到最终的编码为 110101001101。

   所以，对于输入的数据字节 11001101，使用 Hamming编码后的结果为 110101001101

Hamming Code

- 一个问题：Hamming Code的纠错能力上限是什么?
  - 答：一个coded word只能纠错一位!
- 如果错了很多怎么搞?
  - 答：分块Hamming!
    - 所以出现了Hamming(7,4), Hamming(8,4), Hamming(15,11)...之类的分块编码方案!

Hamming Code

Hamming(7,4)

Hamming(7,4) 是一种经典的错误纠正码，它属于 Hamming 码的一种类型。这种编码方法可以将 4 位的数据转换成 7 位的编码，其中包含 4 位原始数据和 3 位校验位。Hamming(7,4) 码能够检测并纠正单个位错误。

在 Hamming(7,4) 编码中，假设原始数据位为 d_1, d_2, d_3, d_4，校验位为 p_1, p_2, p_4（注意校验位的编号通常指示它们在编码序列中的位置，即 2 的幂次）。每个校验位都是几个数据位的异或结果，具体如下：

- p_1 检验位 1，3，5，7 (d_1, d_2, d_3) ；
- p_2 检验位 2，3，6，7 (d_1, d_3, d_4) ；
- p_4 检验位 4，5，6，7 (d_2, d_3, d_4) 。

通过这种方式，任何单一位的错误都会影响独特的校验位组合，从而可以被检测并纠正。

Hamming Code

Hamming(8,4)

通常，Hamming 码是描述成 $(2^r - 1, 2^r - r - 1)$ 的形式，其中 r 是校验位的数量。然而，Hamming(8,4) 并不符合这种典型的描述，因为 8 不是形如 $2^r - 1$ 的数字。因此，Hamming(8,4) 实际上不是一个标准的 Hamming 码。

尽管如此，Hamming(8,4) 可以被理解为一种扩展的 Hamming 码，它可能包含额外的校验位以提供更强的错误检测能力。在这种情况下，一个可能的实现是在标准的 Hamming(7,4) 码基础上添加一个额外的奇偶校验位，使得整个编码块的奇偶性为偶或奇，这样可以增加检测双错误的能力。

例如，如果我们使用标准的 Hamming(7,4) 编码方法得到了一个7位的编码 $c_1, c_2, c_3, c_4, c_5, c_6, c_7$，那么我们可以增加一个额外的位 p_8，它是所有7位的异或结果：

- $p_8 = c_1 \oplus c_2 \oplus c_3 \oplus c_4 \oplus c_5 \oplus c_6 \oplus c_7$

这样，整个编码块将变为 $c_1, c_2, c_3, c_4, c_5, c_6, c_7, p_8$，共8位。如果在传输过程中发生两个错误，那么根据 p_8 的值可能能检测到这种情况，但无法确定具体出错的位置或进行纠正。

循环冗余检验(CRC)

- 循环冗余校验：

  Cyclic Redundancy Check，简称 CRC。

- 通过某种数学运算建立数据和校验位之间的约定关系。
- 编码及译码：
  - 发送端：
    - 被校验数据除以生成多项式；
    - 被校验数据减去余数，结果作为发送数据。
  - 接收端：接收数据除以生成多项式。
    - 可以除尽，编码正确；
    - 除不尽，余数指明出错位所在的位置。

循环冗余检验(CRC)

- 采用模2算术运算：
  - 通过模2减法实现模2除法；
  - 以模2加法将所得余数拼接在被校验数据的后面，形成能除尽的被校验数据。
- 生成多项式应满足的要求：
  - 任何一位发生错误都应使余数不为0；
  - 不同位发生错误应当使余数不同；
  - 应满足余数循环规律。
- 生成多项式的表示：

  如，生成多项式$G=1011_2$，表示生成多项式为

  $G(X)=X^3+X+1$

循环冗余检验(CRC)

【例】

① 假设信息字节为

$F=1001010_2$；

② 选取$G=1011_2$；

③ 将F左移$l-1$位，

形成 $F'=1001010000_2$；

④ 用 F′ 做被除数、G 做除数，

进行模2除法。

忽略商，余数为$R=111_2$。

```
          1010101
      ┌──────────
1011 ) 1001010000
       1011
       ────
        1001
        1011
        ────
         1000
         1011
         ────
          1100
          1011
          ────
           111
```

⑤ 把余数加到 F' 中，组成要发送的信息M：$1001010000_2+111_2=1001010111_2$。

⑥ 接收器采用相反的过程对接收的信息 M 进行解码和校验。M 应该可以被 G 严格整除。

循环冗余检验(CRC)

接收器：解码校验（正确的情况）

【例】

```
              1010101
        ┌──────────
1011 ) 10010101 11
       1011
       ────
        1001
        1011
        ────
         1001
         1011
         ────
          1011
          1011
          ────
          0000
```

接收器：解码校验（1位出错的情况）

```
              1010100
        ┌──────────
1011 ) 10010111 11
       1011
       ────
        1001
        1011
        ────
         1011
         1011
         ────
          0011
```

循环冗余检验(CRC)

- **CRC**的生成多项式的阶数越高，误判的概率就越小。
- 常用的**4**个标准多项式：
  - **CRC-12**：

    $G(X) = X^{12} + X^{11} + X^3 + X^2 + X + 1$

  - **CRC-16(ANSI)**：

    $G(X) = X^{16} + X^{15} + X^2 + 1$

  - **CRC-CCITT(ITU-T)**：

    $G(X) = X^{16} + X^{12} + X^5 + 1$

  - **CRC-32**：

    $G(X) = X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11}$

    $+ X^{10} + X^8 + X^7 + X^5 + X^4 + X^2 + X + 1$

几个小练习

- 2025的IEEE 754浮点如何表示?
- 用奇 and 偶校验编码原码2025?
- 用海明码编码原码2025?
- 用CRC(1011)编码原码2025?